

# Manual — Despliegue y Hosting

TFG DAM — Stockly

Adrián Bravo Santos · Miguel Ángel Florido  
Generado el 2026-05-16



# Guía: hostear el proyecto en internet con un dominio

---

Esta guía cubre **tres caminos** ordenados de menor a mayor dificultad y precio:

1. **Render.com (PaaS, gratis para empezar)** — recomendado si nunca has desplegado antes.
2. **VPS Linux + Nginx + PM2 (DigitalOcean / Hetzner / Contabo)** — el más realista para un TFG, ~5 €/mes.
3. **Docker Compose en VPS** — el más reproducible y "production-ready".

Y en todos los casos: **dominio propio + HTTPS gratis con Let's Encrypt**.

---

## 0. Antes de desplegar

---

- Sube el repo a GitHub: `git remote add origin <url> && git push -u origin main`.
- Genera un `JWT_SECRET` largo:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

- Copia `backend/.env.example` a `.env` real (no se sube al repo).
  - Asegúrate de que `frontend/app.js` usa rutas relativas a `/api`. ✓ Ya lo hace.
- 

## 1. Camino A — Render.com (gratis, ~15 min)

---

### 1.1 Base de datos

Render no incluye MySQL gratis, así que usa una BD externa gratuita:

- **PlanetScale** (MySQL serverless) — <https://planetscale.com>
- O **Aiven** / **Railway MySQL** / **Neon Postgres** (requiere migrar a Postgres).

Crea la BD, importa `db/schema.sql` desde su web o con `mysql -h ... < db/schema.sql`. Apunta:

- `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`.

### 1.2 Web Service para el backend

1. <https://render.com> → **New** → **Web Service** → conecta tu repo.
2. Configuración:

- Root directory: `backend`
- Build command: `npm install`
- Start command: `npm start`
- Plan: `Free`

3. Variables de entorno (pestaña **Environment**): pega todas las del `.env`.

4. Deploy → al cabo de unos minutos te da una URL `https://stockly.onrender.com`.

### 1.3 Servir el frontend

El backend ya sirve `frontend/` con `express.static`, así que la misma URL muestra el login. ✓

### 1.4 Dominio propio

1. Compra el dominio (recomendado: **Namecheap**, **Porkbun** o **IONOS**, ~10 €/año).
2. En Render → **Settings** → **Custom domain** → añade `tfg.tudominio.com`.
3. Render te indica un registro CNAME → cópialo en el panel DNS de tu registrador.
4. HTTPS se emite automáticamente (Let's Encrypt) en cuanto el DNS propaga.

---

## 2. Camino B — VPS Linux con Nginx + PM2

---

Recomendado para el TFG: te enseña administración real de servidor.

Proveedores baratos:

- **Hetzner Cloud CX22**: ~4 €/mes (Helsinki / Núremberg).
- **DigitalOcean Droplet**: 5 \$/mes.
- **Contabo VPS S**: 5 €/mes (más recursos).

### 2.1 Crear servidor y conectarse

```
ssh root@TU.IP.DEL.VPS
```

Sistema recomendado: **Ubuntu 22.04 LTS**.

## 2.2 Setup base

```
apt update && apt -y upgrade
apt -y install nginx git curl ufw mariadb-server certbot python3-certbot-nginx
ufw allow OpenSSH && ufw allow 'Nginx Full' && ufw enable

# Node 20 LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt -y install nodejs
npm i -g pm2
```

## 2.3 Base de datos

```
mysql_secure_installation      # define password de root, etc.
mysql -u root -p < /tmp/schema.sql  # subido por scp / git clone
```

## 2.4 Clonar y arrancar el backend

```
adduser tfg
su - tfg
git clone https://github.com/<tu-usuario>/TFGDAM.git
cd TFGDAM/backend
cp .env.example .env
nano .env                      # rellena credenciales reales + JWT_SECRET
npm ci
pm2 start server.js --name tfg-api
pm2 save
sudo env PATH=$PATH:/usr/bin pm2 startup systemd -u tfg --hp /home/tfg
```

## 2.5 Nginx como reverse proxy

Crea `/etc/nginx/sites-available/tfg`:

```
server {
    listen 80;
    server_name tfg.tudominio.com;

    client_max_body_size 5m;
    gzip on;
    gzip_types text/css application/javascript image/svg+xml application/json;

    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
ln -s /etc/nginx/sites-available/tfg /etc/nginx/sites-enabled/  
nginx -t && systemctl reload nginx
```

## 2.6 Dominio + HTTPS

1. En el panel DNS del registrador, crea un registro **A**:
  - **Tipo:** A · **Host:** tfg · **Valor:** TU.IP.DEL.VPS · TTL 3600.
2. Espera que `dig tfg.tudominio.com` apunte a tu IP (1-30 min).
3. Emite certificado:

```
certbot --nginx -d tfg.tudominio.com
```

Certbot añade el bloque `listen 443 ssl` y configura redirección 80→443. Renovación automática vía cron.

## 2.7 Backups básicos

Crea `/etc/cron.daily/backup-tfg`:

```
#!/bin/bash  
DATE=$(date +%F)  
mysqldump --single-transaction stockly | gzip > /var/backups/tfg-$DATE.sql.gz  
find /var/backups -name 'tfg-*.sql.gz' -mtime +14 -delete
```

```
chmod +x /etc/cron.daily/backup-tfg
```

---

## 3. Camino C — Docker Compose en VPS

Crea en la raíz del repo `docker-compose.yml`:

```

services:
  db:
    image: mariadb:10.11
    restart: always
    environment:
      MARIADB_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MARIADB_DATABASE: stockly
    volumes:
      - dbdata:/var/lib/mysql
      - ./db/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro
  api:
    build: ./backend
    restart: always
    environment:
      DB_HOST: db
      DB_USER: root
      DB_PASSWORD: ${DB_ROOT_PASSWORD}
      DB_NAME: stockly
      JWT_SECRET: ${JWT_SECRET}
      NODE_ENV: production
    depends_on: [db]
    ports: ["3001:3001"]
  volumes:
    dbdata:

```

Y `backend/Dockerfile` :

```

FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
COPY ../frontend ../frontend
EXPOSE 3001
CMD ["node", "server.js"]

```

Nota: con Docker es más limpio mover `frontend/` dentro de `backend/public` durante el build.

Comandos:

```

docker compose up -d --build
docker compose logs -f api

```

Combina con Nginx + Certbot del paso 2.5/2.6 para HTTPS.

## 4. Comprobaciones tras desplegar

```
curl https://tfg.tudominio.com/api/health
# { "ok": true, "db": true, "ts": "..."} }
```

- ¿Login funciona? Abre el dominio en el navegador.
- ¿HTTPS válido? Candado en la barra de direcciones.
- ¿PWA instalable? Chrome → menú → "Instalar Stockly".
- ¿Logs limpios? `pm2 logs tfg-api --lines 50` o `docker compose logs --tail=50 api`.

## 5. Buenas prácticas de seguridad mínimas

- Cambia `JWT_SECRET` y las contraseñas semilla antes de exponer la app.
- Crea un usuario MySQL específico (no root) y dale permisos sólo sobre `stockly`.
- Configura **fail2ban** en el VPS (`apt install fail2ban`) para bloquear intentos SSH.
- Mantén `apt -y upgrade` semanal y `npm audit fix` periódico.
- Habilita **HSTS** en Nginx tras verificar que el HTTPS funciona estable.

## 6. Coste real estimado para 1 año

| Concepto                                     | Coste anual aprox. |
|----------------------------------------------|--------------------|
| Dominio <code>.com</code> / <code>.es</code> | 10-12 €            |
| VPS Hetzner CX22                             | ~50 €              |
| Backups objeto (Hetzner / Backblaze)         | 5-10 €             |
| <b>Total</b>                                 | <b>~65-75 €</b>    |

Render.com gratis es 0 € pero el servicio se "duerme" tras 15 min de inactividad; sirve para defensa pero no para uso real continuo.